

Noventra

Autonomous Liquidity Orchestration via Multi-Agent AI on Somnia

Technical Whitepaper | Agentathon 2025

Krishna Goyal <krishnagoyalcse@gmail.com>

<https://github.com/krishnagoyal099/Noventra>

June 2025

Abstract

Noventra is an autonomous liquidity orchestration protocol that deploys a swarm of five specialised AI agents — Scout, Risk, Strategy, Execution, and Coordinator — on the Somnia Testnet (EVM-compatible, Chain ID 50312). Each agent holds a separate funded wallet and communicates exclusively through an immutable on-chain MessageBus. The agents collaborate to identify DeFi yield opportunities, evaluate risk, propose allocation strategies via Groq LLM reasoning, and execute intent-driven trades through a Flashbots SUAVE-inspired solver architecture. All agent decisions, reasoning traces, and execution proofs are stored as verifiable on-chain receipts, creating a fully auditable, autonomous DeFi portfolio manager that requires no human intervention after deployment.

Contents

1	Introduction	3
2	System Architecture	3
2.1	Layer Overview	3
2.2	Agent Pipeline	3
3	Smart Contracts	3
3.1	ALOCore	3
3.2	MessageBus	4
3.3	AgentRegistry	4
3.4	Deployed Addresses	5
4	Agent Specifications	5
4.1	ScoutAgent — Dual-Mode Yield Detector	5
4.1.1	Mode 1: Passive Yield Scanning	5
4.1.2	Mode 2: JIT Mempool Predator	5
4.2	RiskAgent — Statistical Guardrails	5
4.3	StrategyAgent — LLM-Powered Intent Originator	6
4.4	ExecutionAgent — Intent-Centric Solver	6
4.5	CoordinatorAgent — Treasury and Deadlock Monitor	6
5	Intent-Centric Architecture	7
6	Somnia-Specific Engineering	7
6.1	Known RPC Constraints and Mitigations	7
6.2	Custom SomniaProvider	8
6.3	Chunked queryFilter	8
7	Signal Schemas	9
8	Frontend Dashboard	9

9 Security	9
9.1 On-Chain Controls	9
9.2 Off-Chain Controls	10
10 Performance	10
11 Comparison	10
12 Deployment	10
13 Roadmap	11
14 Conclusion	11

Introduction

Traditional DeFi protocols rely on rigid smart-contract logic or centralised off-chain bots that operate in silos, expose users to MEV, and cannot adapt to changing market conditions without redeployment. Human portfolio managers face liquidity fragmentation across dozens of protocols, while automated bots typically optimise for a single metric (highest APY) without holistic risk evaluation.

Noventra addresses this through a multi-agent system (MAS) architecture where:

- Each agent is a specialised autonomous actor with its own on-chain identity (wallet).
- Inter-agent communication is fully on-chain via an ABI-encoded MessageBus.
- Strategic decisions incorporate LLM reasoning (Groq / Llama-3) stored immutably as on-chain reasoning traces.
- Execution follows an intent-centric solver model inspired by Flashbots SUAVE: agents express *what* they want, not *how* to achieve it.

Somnia’s high-throughput EVM chain (400k TPS target, ~1 second block time) is uniquely suited for this architecture. Agents can coordinate through on-chain messages at latencies comparable to centralised APIs, making the MessageBus viable as a real-time coordination layer.

System Architecture

Layer Overview

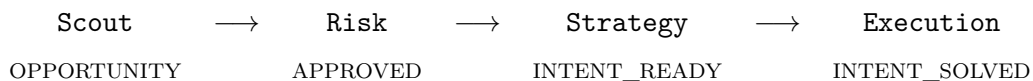
The Noventra system is structured across three layers:

Table 1: System Layer Summary

Layer	Components	Responsibility
On-chain	ALOCore, MessageBus, AgentRegistry	State, coordination, receipts
Agent Swarm	Scout, Risk, Strategy, Execution, Coordinator	Autonomous decision-making
Frontend	React dashboard, useSomnia hook	User interface, intent submission

Agent Pipeline

Agents communicate through a directed acyclic graph (DAG) enforced by the CoordinatorAgent. No agent can proceed without the previous stage completing successfully on-chain.



Each arrow represents an on-chain **SignalSent** event emitted to the MessageBus. Every signal is ABI-encoded, timestamped, and permanently stored, creating a complete audit trail of every autonomous decision.

Smart Contracts

ALOCore

The **ALOCore** contract stores all strategy proposals, execution receipts, and manages total system liquidity. Its key data structures are:

Listing 1: ALOCore Strategy Data Structures

```

1 struct StrategyParams {
2     address targetPool;
3     uint256 allocation;
4     uint256 expectedAPY;
5     string rationale; // LLM reasoning trace stored immutably
6 }
7
8 struct Strategy {
9     address proposer;
10    StrategyParams params;
11    bool approved;
12    uint256 timestamp;
13 }
14
15 event StrategyProposed(uint256 indexed id, address indexed proposer);
16 event StrategyEvaluated(
17     uint256 indexed id,
18     address evaluator,
19     bool approved,
20     string reason
21 );

```

MessageBus

The MessageBus is the nervous system of the swarm. Every inter-agent message is an ABI-encoded payload stored permanently on-chain.

Listing 2: MessageBus Signal Interface

```

1 event SignalSent(
2     bytes32 indexed signalId,
3     address indexed by,
4     string signalType, // "OPPORTUNITY_FOUND", "INTENT_READY", etc.
5     bytes data,        // ABI-encoded payload
6     uint256 timestamp
7 );
8
9 function emitSignal(string calldata signalType, bytes calldata data)
10     external returns (bytes32 signalId);
11
12 function consumeSignal(bytes32 signalId) external;

```

AgentRegistry

Maintains an on-chain registry of authorised agent wallets, keyed by role. Only registered agents may emit signals or propose strategies.

Listing 3: AgentRegistry Role Enforcement

```

1 enum AgentRole { SCOUT, RISK, STRATEGY, EXECUTION, COORDINATOR }
2
3 function registerAgent(address agent, AgentRole role) external onlyOwner
4     ;
5 function isRegistered(address agent) external view returns (bool);
6 function getAgentsByRole(AgentRole role)
7     external view returns (address[] memory);

```

Deployed Addresses

Table 2: Contract Addresses on Somnia Testnet (Chain ID 50312)

Contract	Address
ALOCore	0x2beE183Ce98cEBf13dfcaeba4c2965Cda8cDd13d
MessageBus	0x1e326DeC20fa7006C827e2DA6fa8dc56e7276d21
AgentRegistry	0x332CAd34b8291ACfAD87c8A6A41126cde3F906a6
MockDEX	0x770082BcfC03824e47E6eC63afcAb0C6866cFf27
YieldPool Alpha	0xf22AC8A79C638aD8B92861dF9f32b241599E9C02
YieldPool Beta	0x369797e23813775069Ef41ad3a3519C1ccBaC003

Agent Specifications

ScoutAgent — Dual-Mode Yield Detector

The ScoutAgent runs two parallel detection engines.

Mode 1: Passive Yield Scanning

Polls `MockYieldSource.getCurrentAPY()` every 30 seconds across all registered pools. Emits `OPPORTUNITY_FOUND` when:

$$\text{APY}_{\text{current}} \geq \text{threshold} \quad \text{AND} \quad \text{APY}_{\text{current}} > \text{APY}_{\text{previous}}$$

The momentum filter (requiring rising APY) prevents spurious signals during transient spikes.

Mode 2: JIT Mempool Predator

Listens for `MockDEX.LargeSwapPending` events, which serve as an on-chain proxy for `eth_subscribe pendingTransactions`. When a large swap is detected, the Scout calculates a concentrated tick range and emits `JIT_OPPORTUNITY_FOUND`:

$$[\text{tickLower}, \text{tickUpper}] = [\text{tick}_{\text{current}} - 5, \text{tick}_{\text{current}} + 5]$$

$$\text{feeEfficiency} = \frac{\text{estimatedFeeCapture}}{\text{amountIn}} \times 10,000 \text{ bps}$$

Capital is deployed for exactly one block, eliminating impermanent loss exposure.

RiskAgent — Statistical Guardrails

The RiskAgent evaluates every proposed strategy before allowing execution.

Table 3: Risk Evaluation Criteria

Criterion	Threshold	Action on Breach
APY ceiling	$\leq 10,000 \text{ bps}$	Reject (likely fraudulent)
Maximum drawdown	$\leq 20\%$	Reject
Allocation per strategy	$\leq 30\%$	Clamp and warn

StrategyAgent — LLM-Powered Intent Originator

The StrategyAgent incorporates large language model reasoning as a first-class component. Upon receiving an `OPPORTUNITY_FOUND` or `USER_INTENT` signal, it:

1. Constructs a structured prompt containing the opportunity data, current liquidity, and risk constraints.
2. Queries Groq (Llama-3.3-70b) in JSON mode, receiving:

Listing 4: LLM Strategy Decision Schema

```

1 interface LLMStrategyDecision {
2   allocation_percentage: number; // 0-100; code enforces hard cap
3   risk_assessment: string;       // one-sentence internal log
4   rationale: string;             // stored permanently on-chain
5 }
```

3. Stores the `rationale` field as an immutable on-chain receipt in `ALOCore` — making the AI’s reasoning verifiable and auditable.
4. Emits `INTENT_READY` to the MessageBus, encoding *what* the system wants (pool, allocation, target APY) without prescribing *how* to execute it.

ExecutionAgent — Intent-Centric Solver

Inspired by the Flashbots SUAVE architecture, the ExecutionAgent acts as a Solver rather than a simple executor. It:

1. Polls the MessageBus for `INTENT_READY` signals every 10 seconds.
2. Independently simulates two execution routes off-chain:
 - **Route A:** Single-hop direct swap on DEX-A (full amount).
 - **Route B:** 60/40 split across DEX-A and DEX-B (aggregator bonus applied to the DEX-B leg).
3. Selects the route that maximises output (minimises effective slippage):

$$\text{slippageBps} = \frac{(\text{amount} - \text{routeOutput}) \times 10,000}{\text{amount}}$$

4. Emits `INTENT_SOLVED` with ABI-encoded proof of the optimisation: (`routeDescription`, `slippageBps`, `mevProtected`).

CoordinatorAgent — Treasury and Deadlock Monitor

The CoordinatorAgent serves two operational roles:

DAG Enforcer. Monitors all agents on a configurable heartbeat interval. If any agent stops responding within a 45-second timeout, it emits an `AGENT_TIMEOUT` signal and initiates recovery.

Swarm Gas Station. Acts as a shared treasury. Users fund the Coordinator’s wallet via the frontend dashboard. The Coordinator automatically redistributes STT to any agent wallet that drops below the refill threshold:

Listing 5: Coordinator Auto-Refill Logic

```

1 private async refillAgentIfNeeded(agentSigner: Signer): Promise<void> {
2   const addr      = await agentSigner.getAddress();
3   const balance = await this.wallet.provider!.getBalance(addr);
4   if (balance < this.REFILL_THRESHOLD) {           // < 1 STT
5     const tx = await this.wallet.sendTransaction({
6       to:      addr,
7       value:    this.REFILL_AMOUNT,                // 1 STT
8       gasLimit: 21_000n,
9     });
10    await tx.wait();
11    this.logSuccess(`Refilled ${addr.slice(0, 10)} with 1 STT`);
12  }
13 }

```

Intent-Centric Architecture

A central design principle distinguishes Noventra from conventional on-chain bots:

*Traditional agents prescribe **how** to execute. Noventra agents express **what** outcome they desire.*

The StrategyAgent issues an `INTENT_READY` signal specifying the target pool, allocation amount, and expected APY. The ExecutionAgent (Solver) independently determines the optimal route, MEV protection strategy, and split ratios. This separation:

- Enables multiple solvers to compete on execution quality in future versions.
- Allows route optimisation logic to evolve independently of strategy logic.
- Creates verifiable on-chain proof of solver work, not just execution outcomes.

This architecture mirrors the Flashbots SUAVE intent-expression model, adapted for agent-to-agent coordination rather than user-to-block-builder interaction.

Somnia-Specific Engineering

Known RPC Constraints and Mitigations

Somnia Testnet exhibits several non-standard EVM behaviours that required engineering workarounds:

Table 4: Somnia Testnet Constraints and Applied Fixes

Constraint	Impact	Fix
eth_getLogs returns removed: null	ethers.js v6 throws a BAD_DATA assertion error	Custom SomniaProvider normalises null → false
Gas estimation returns ~2.2B gas	Transactions fail as invalid	All gasLimit values capped at 12M; maxFeePerGas set explicitly
eth_getLogs block range limit: 1000 blocks	History queries silently fail	Chunked queryFilter in pages of 999 blocks
Storage array push() reverts	ALOCore array appends fail on Somnia	Bypass via MessageBus.emitSignal; persistent arrays avoided
HTTP-only RPC, no WebSocket	contract.on() never fires	Poll-based setInterval + queryFilter replaces all listeners

Custom SomniaProvider

Listing 6: SomniaProvider with Two Quirk Fixes

```

1 class SomniaProvider extends JsonRpcProvider {
2   private static readonly GAS_CAP = 12_000_000n;
3
4   override async send(method: string, params: Array<any>): Promise<any>
5   {
6     const result = await super.send(method, params);
7
8     // Fix 1: Normalise log.removed null -> false
9     if (method === "eth_getLogs" && Array.isArray(result)) {
10       return result.map((log: any) => ({
11         ...log,
12         removed: log.removed ?? false,
13       }));
14     }
15
16     // Fix 2: Cap inflated gas estimates
17     if (method === "eth_estimateGas") {
18       const estimated = BigInt(result);
19       if (estimated > SomniaProvider.GAS_CAP)
20         return "0x" + SomniaProvider.GAS_CAP.toString(16);
21     }
22
23     return result;
24   }
25 }

```

Chunked queryFilter

Listing 7: Chunked queryFilter Respecting Somnia's 1000-Block Limit

```

1 const queryChunked = async (filter: any, from: number, to: number) => {
2   const CHUNK = 999;
3   const all: any[] = [];

```



```

4   for (let start = from; start <= to; start += CHUNK) {
5       const end    = Math.min(start + CHUNK - 1, to);
6       const chunk = await messageBus.queryFilter(filter, start, end);
7       all.push(...chunk);
8   }
9   return all;
10 };

```

Signal Schemas

Every on-chain signal has a defined ABI encoding schema:

Table 5: MessageBus Signal Types and Payload Schemas

Signal Type	ABI-Encoded Payload
OPPORTUNITY_FOUND	(address pool, uint256 apy, uint256 detectedAt, string name)
STRATEGY_PROPOSED	(uint256 strategyId, string rationale, uint256 allocation)
INTENT_READY	(uint256 strategyId, address pool, uint256 allocation, uint256 apy)
INTENT_SOLVED	(string routeDescription, uint256 slippageBps, bool mevProtected)
JIT_OPPORTUNITY_FOUND	(address tokenIn, address tokenOut, uint256 amount, int24 tickLower, int24 tickUpper, uint256 feeCapture, uint256 blockV)

Frontend Dashboard

The Noventra frontend is a React single-page application deployed on Vercel. It provides:

Live Agent Terminal.

Displays the 15 most recent MessageBus signals in real time. Uses the chunked `queryFilter` pattern to load up to 3000 blocks of history on page load, then polls every 10 seconds for new events.

Natural Language Intent Submission.

Users type free-form intents (e.g. “Maximise yield conservatively”) which are ABI-encoded and broadcast as `USER_INTENT` signals. The StrategyAgent intercepts these within 12 seconds and passes them to the LLM for interpretation.

Fund Swarm.

Sends STT directly to the Coordinator’s treasury wallet via MetaMask. A minimum of 1 STT per agent (5 STT total) is sufficient for several hours of autonomous operation.

Portfolio View.

Shows per-user deposits and simulated yield accumulated across YieldPool Alpha and Beta.

Security

On-Chain Controls

- **Role-based access.** Only addresses registered in `AgentRegistry` may emit signals or propose strategies. The registry is owner-controlled.
- **Allocation caps.** The StrategyAgent enforces a configurable maximum allocation percentage (10% in the current testnet deployment) regardless of LLM output.

- **Risk veto.** The RiskAgent rejects any strategy with APY exceeding 100% or drawdown exceeding 20%, providing a hard guard against malicious or hallucinated inputs.

Off-Chain Controls

- Private keys are stored in environment variables and are never committed to version control.
- The production Docker container runs as a non-root user (**noventra**).
- All inter-agent communication is on-chain — there is no trusted off-chain channel that could be intercepted or spoofed.

Performance

Table 6: Observed System Performance on Somnia Testnet

Metric	Observed Value	Notes
End-to-end latency (Opportunity → Solved)	25–60 seconds	5-signal DAG
Scout scan interval	30 seconds	Prevents nonce collisions
StrategyAgent poll cadence	12 seconds	MessageBus query
ExecutionAgent poll cadence	10 seconds	Intent pickup
Gas per <code>emitSignal</code> call	100k–500k gas	Payload-dependent
Estimated gas cost per full cycle	0.003–0.01 STT	At 6 Gwei

Comparison

Table 7: Feature Comparison with Related Systems

Feature	Noventra	Yearn v3	Flashbots	AutoGPT
Multi-agent swarm	Yes	No	No	Yes
On-chain agent coordination	Yes	No	No	No
LLM reasoning trace	Yes	No	No	Yes
Intent-centric solver	Yes	No	Yes	No
MEV protection	Yes	No	Yes	No
Verifiable on-chain proofs	Yes	No	No	No
Autonomous gas management	Yes	No	No	No

Deployment

Agent Swarm.

Docker image deployed to Railway. Entry point: `node -require ts-node/register scripts/listen-somnia.ts`. The image copies the `deployments/` directory (containing `somnia.json`) to ensure contract addresses are available at runtime.

Frontend.

Vite/React application deployed to Vercel via GitHub push-triggered CI/CD.

Contracts.

Deployed to Somnia Testnet using Hardhat and verified on the Shannon Explorer (<https://shannon-explorer.somnia.network>).

Roadmap

Phase 1 — Testnet MVP (current).

Five-agent swarm on Somnia Testnet with LLM reasoning, on-chain receipts, intent-centric execution, and live frontend dashboard.

Phase 2 — Real Protocol Integration.

Replace MockDEX and MockYieldSource with live Uniswap V3, Aave, and Compound adapters. Integrate the Flashbots Protect RPC for genuine MEV shielding.

Phase 3 — Agent Marketplace.

Open the AgentRegistry to third-party agents via a standardised **IAgent** interface. External developers will deploy specialised sub-agents (e.g. cross-chain bridging) that plug into the existing coordination DAG.

Phase 4 — Mainnet and Tokenomics.

Native governance token. Staking-based access control for premium strategies. On-chain DAO vote for risk parameter updates.

Conclusion

Noventra demonstrates that autonomous, AI-coordinated DeFi is practically deployable on a real high-performance EVM chain today. By combining an on-chain MessageBus for verifiable agent coordination, LLM-powered reasoning with immutable on-chain traces, an intent-centric solver architecture, and a self-funding treasury for autonomous gas management, Noventra establishes a blueprint for the next generation of DeFi protocols: ones that are not merely automatic, but genuinely autonomous.

The full source code is available at:

<https://github.com/krishnagoyal099/Noventra>

References

1. Wan, Z., & Adams, H. (2024). *Just-in-Time Liquidity in Uniswap V3*. Uniswap Labs Research.
2. Yu, S. et al. (2024). *FinAgent: A Multimodal Foundation Agent for Financial Trading*. arXiv:2402.18485.
3. Flashbots. (2023). *SUAVE: Single Unifying Auction for Value Expression*. <https://writings.flashbots.net/the-future-of-mev-is-suave>
4. Somnia Network. (2024). *Technical Overview*. <https://docs.somnia.network>
5. Buterin, V. (2014). *A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum Whitepaper.
6. Brown, T. et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS 2020.